

System Architecture Basics

From Monolith to Microservices

Rasala Kadhiraiah

Senior Enterprise Architect & Technical Trainer

1. The Monolithic Approach

The "Single Block" Era

In the beginning, your application is a single, unified unit.

Why it works (The Pros):

- ✓ **Simple Start:** Fast to build and deploy.
- ⚡ **High Speed:** Modules talk to each other instantly (zero network lag).
- 🛠️ **Easy Debug:** Trace a bug from UI to Database in one place.

1. The Monolith (Continued)

The Growth Pain

As your business grows, the "Single Block" becomes a bottleneck.

The Bottlenecks:

-  **Heavy Scaling:** Must duplicate the *whole* app even to scale one small feature.
-  **Tight Coupling:** Change one line of code, risk breaking the entire system.
-  **Slow Deploy:** Huge codebase means 30-minute build times.

2. Microservices Architecture



The Modular Revolution

Breaking the block into autonomous, specialized services.



Independent Scaling

Scale only what you need (e.g., only the *Payment* service during Black Friday).



Fault Tolerance

If the *Inventory* service crashes, customers can still *Browse* items.



Tech Freedom

Use **Java** for core logic, **Node.js** for real-time chat, and **Python** for AI.



Fast Delivery

Small teams deploy small services daily without waiting for others.

3. API Gateways

The Single Entry Point

In a world of 50 microservices, the client shouldn't have to track 50 URLs.

-  **Traffic Cop:** Routes requests to the correct service.
-  **Security Guard:** Centralizes JWT Authentication & Rate Limiting.
-  **Protocol Translator:** Converts external HTTP calls to internal gRPC or WebSockets.

4. Data Management

From One to Many

The Anti-Pattern

Shared Database:

Services fighting over the same tables leads to "Distributed Monolith" hell.

The Gold Standard

Database-per-Service:

Every service owns its data. Use **Redis** for speed and **PostgreSQL** for safety.

5. CI/CD & Automation

The Engine Room

You cannot manage microservices manually. You need a pipeline.

1.  **Containerize:** Use **Docker** so it runs everywhere.
2.  **Auto-Test:** Run **JUnit** & **Mockito** on every commit.
3.  **Deploy:** Use **Jenkins** to ship code 24/7 with zero downtime.

Questions?

Let's build something scalable.

GitHub Portfolio: `github.com/rkadhiraiah`

Email: `rkadhiraiah@gmail.com`